LEADERS UNIVERSITY / LICENCE BI

Indices et corrigés

Atlas puis CampusBox · comprendre une correction, garder sa tentative

28 h	70 %	0
AU TOTAL	PRATIQUE	PRÉREQUIS IT

ATLAS PAPETERIE

Fichiers de ventes → ETL → Entrepôt → Rapport

05 / INDICES ET CORRIGÉS

Youssef Douma · Leaders University

Édition pédagogique · 27 octobre 2026 · données fictives

INDICES / TROIS DEGRÉS D'AIDE

Choisir une aide sans sauter l'expérience

Commencer par votre prédiction et une tentative enregistrée. Lire ensuite le premier indice de la mission. Si le blocage persiste, lire le second indice puis comparer à la référence complète séparée. Les chemins de correction ne sont pas une consigne de remplacer tout le projet dès le départ.

Aide	Usage
Indice 1 · sens	Retrouver le grain, l'unité ou la responsabilité d'une étape.
Indice 2 · expérience	Réduire à un petit cas et observer le point de divergence.
Référence	Comparer une fonction précise et rejouer la variation.
Reprise	Créer une nouvelle copie du projet terminée jusqu'au jalon choisi.

Les fonctions complètes sont dans solutions/atlas/. Vos fichiers de travail sont dans projet/. Le vérificateur public ne lit jamais solutions/atlas/ pour juger votre fonction ; il contrôle les comportements et les sorties réellement produits.

Une correction doit vous apprendre quelque chose

Après consultation, noter ce qui manquait, le test qui l'a révélé et une variation que vous pouvez expliquer. Un résultat copié sans explication ne vous prépare pas au prochain cas.

INDICES / COMMENCER PETIT

Indices 01 et 02 : calculer puis lire

01 · indice de sens

Le prix unitaire n'est pas le montant. Chaque vente contribue quantity × unit_price_millimes. Le total commence à zéro et s'accumule ; return rend la somme.

01 · expérience réduite

Avec une seule vente de quantité 4 et prix 125 millimes, attendre 500, pas 125. Avec aucune vente, attendre 0. Si le programme renvoie 6000 pour le panier initial, il a ignoré les quantités.

02 · indice de sens

Le lecteur conserve les observations ; le validateur décidera plus tard. Les valeurs CSV restent du texte, y compris « 001 ». L'en-tête n'est pas une vente.

02 · expérience réduite

Tester un CSV avec un accent, une virgule entre guillemets et un code « 001 ». csv.DictReader gère ces cas ; découper chaque ligne avec split(",") ne respecte pas les champs entre guillemets.

Avant de regarder le code corrigé, montrer l'entrée et la sortie attendue avec vos propres mots. Vérifier que vous avez remplacé seulement la fonction TODO puis enregistré le vrai fichier .py.

INDICES / QUALITÉ ET MODÈLE

Indices 03 et 04 : décider puis relier

03 · indice de sens

Écrire une issue pour chaque ligne : retenue, rejet avec motif ou doublon. Normaliser avant de comparer les contenus. Un identifiant dont le contenu change est un conflit explicite.

03 · expérience réduite

Une vente de 2 articles à 0.125 TND doit produire amount_millimes=250. Ajouter sa répétition et une quantité zéro : 1 retenue, 1 doublon, 1 rejet. Le motif du rejet doit être non vide.

04 · indice de sens

Le fait représente une vente ; les dimensions donnent son contexte. La clé produit est cherchée par product_id. La clé étrangère du fait doit pointer vers une ligne existante.

04 · expérience réduite

Créer une base neuve, compter ses quatre tables et ses zéros lignes. Relancer la création : les tables restent là. Sur une base d'essai remplie, la création ne doit jamais supprimer les données.

Si le module 03 ne produit pas ses CSV, garder main() et ecrire_csv() fournis : remplacer tout le fichier par une seule fonction aurait supprimé le lancement. Le contrôle ne crée pas les CSV de votre projet à votre place.

INDICES / COMPORTEMENT COMPLET

Indices 05, 06 et 07 : charger puis expliquer

05 · regarder l'identité

Comparer la vente présente à la vente entrante sur date, produit, boutique, quantité et prix. Si elle est identique, compter existing. Si elle diffère, lever un conflit et annuler le lot entier, y compris une vente insérée juste avant.

06 · compter à deux niveaux

Trois lignes valides dans le fichier d'ajout ne font que deux nouvelles ventes en base. Les comptes raw/accepted/rejected/duplicates concernent le fichier ; inserted/existing concernent la base.

07 · partir du vrai SQL

Le rapport lit la base actuelle. COUNT et SUM donnent les mesures ; MIN et MAX des dates donnent la période. Le titre et les unités doivent décrire ces résultats, pas les prédire ou les remplacer.

Pour un défaut de rejeu, relire charger avant de réécrire ajouter. Pour un HTML incohérent, comparer la requête et le dictionnaire renvoyé avant de retoucher le style. Conserver les helpers fournis réduit les interfaces à modifier.

Si l'aide ne suffit pas

Reproduire le défaut avec une entrée de deux ou trois lignes, écrire attendu/obtenu, puis consulter la fonction de référence correspondante. La page de reprise en fin de ce livre permet de poursuivre depuis une copie indépendante.

CORRIGÉS / À LIRE APRÈS LA TENTATIVE

Correction 01 : multiplier avant d'additionner

La référence calcule un montant par vente puis l'ajoute au total. La liste vide renvoie naturellement zéro. Les valeurs d'essai ne sont jamais inscrites comme réponse fixe dans la fonction.

Python · version courte à comparer à calcul.py

```
def calculer_total(ventes):
    total = 0
    for vente in ventes:
        quantite = vente["quantity"]
        prix = vente["unit_price_millimes"]
        total = total + quantite * prix
    return total
```

Cas	Résultat
2 carnets + 3 stylos	13 500 millimes
3 carnets + 3 stylos	18 000 millimes
Liste vide	0 millime

La version complète commentée est dans solutions/atlas/calcul.py. Ne remplacer que la définition calculer_total de votre fichier. Garder le main() fourni qui affiche le panier puis sa variation.

PowerShell / Ubuntu · racine du cours

```
docker compose run --rm atelier python calcul.py
docker compose run --rm atelier python verifier.py 01
```

Question après correction

Si le prix double pour un produit, quelle contribution double ? Pourquoi le nombre de lignes ne change-t-il pas ?

CORRIGÉS / EXTRACTION FIDÈLE

Correction 02 : garder les champs et le texte

Python · extrait, imports csv et Path déjà fournis

```
def lire_csv(chemin):
    with Path(chemin).open(
        encoding="utf-8-sig", newline=""
    ) as fichier:
        return list(csv.DictReader(fichier))
```

Ce code est un extrait à lire dans son module, pas un programme autonome. utf-8-sig accepte aussi un éventuel BOM au début du fichier. newline="" laisse le module CSV gérer correctement les fins de lignes. La liste est construite pendant que le fichier est encore ouvert.

Constat	Sens
16 lignes initiales	Toutes les observations arrivent au nettoyage.
3 lignes d'ajout	Le lecteur n'a pas encore décidé ce qui est nouveau.
quantity reste str	Le validateur vérifiera avant la conversion.

Comparer uniquement lire_csv avec solutions/atlas/lecture.py. Conserver ROOT, DATA, OUTPUTS, catalogues(), ecrire_csv() et main() présents dans le starter. Le contrat est réutilisé par toutes les missions suivantes.

PowerShell / Ubuntu · racine du cours

```
docker compose run --rm atelier python lecture.py
docker compose run --rm atelier python verifier.py 02
```

CORRIGÉS / AUCUNE LIGNE DISPARUE

Correction 03 : réconcilier la qualité

Sortie du lot initial	Résultat
outputs/ventes_nettoyees.csv	12 ventes, 32 articles, 99 000 millimes.
outputs/rejets.csv	3 lignes avec les six champs source et motif.
outputs/doublons.csv	1 répétition avec les champs normalisés.

La référence normalise chaque ligne, valide les valeurs, calcule les deux montants entiers puis compare l'identifiant aux ventes déjà vues. Le dictionnaire des identifiants vus contient leur contenu normalisé. Une ligne invalide va dans les rejets ; une contradiction d'identité interrompt explicitement le lot.

- Normalisation : enlever les espaces et uniformiser la casse des codes.
- Prix : Decimal, valeur finie positive, nombre entier de millimes.
- Rejet : garder les valeurs reçues et le motif pour permettre l'inspection.
- Doublon : même contenu normalisé ; il ne doit pas augmenter le montant.

La référence complète est dans solutions/atlas/nettoyage.py. Elle sépare une validation locale normaliser() pour la lisibilité. Ce helper appartient à cette référence ; votre fonction nettoyer peut être structurée autrement si elle respecte la même entrée et les mêmes sorties.

PowerShell / Ubuntu · racine du cours

```
docker compose run --rm atelier python nettoyage.py
docker compose run --rm atelier python verifier.py 03
```

Question après correction

16 = 12 + 1 + 3 : montrer dans les fichiers une ligne qui justifie chacune des trois issues.

CORRIGÉS / LE MODÈLE EN ÉTOILE

Correction 04 : quatre tables sans effacement

Table	Clé et contenu essentiel
dim_product	product_key PK ; product_id unique ; product_name.
dim_shop	shop_key PK ; shop_id unique ; shop_name.
dim_date	date_key PK ; full_date unique ; year, month, day.
fact_sales	sale_id PK ; trois FK ; quantity ; prix et montant en millimes.

La référence complète de solutions/atlas/modele.py utilise CREATE TABLE IF NOT EXISTS pour rendre la création relançable. Les contraintes de quantité, de prix et de montant rendent des états faux plus difficiles à enregistrer. La fonction creer_modele ne fait aucun INSERT.

Le helper connecter() active PRAGMA foreign_keys=ON à chaque connexion. L'activer une seule fois dans un autre programme ne suffit pas : ce réglage appartient à la connexion SQLite. Le schéma porte trois relations depuis le fait vers les dimensions.

PowerShell / Ubuntu · racine du cours

```
docker compose run --rm atelier python modele.py
docker compose run --rm atelier python verifier.py 04
```

Question après correction

Pourquoi une base vide et correcte contient-elle les quatre tables mais aucun total de ventes ? Montrer quelle mission doit la remplir.

CORRIGÉS / TRANSACTION ET CONTRÔLES SQL

Correction 05 : charger le lot et le protéger

La référence de solutions/atlas/chargement.py suppose la base créée. Elle travaille dans une transaction, retrouve les dimensions par leurs codes et compare le contenu des faits existants. Un conflit déclenche le retour arrière de tout le lot.

Contrôle initial	Attendu
Faits / articles / montant	12 / 32 / 99 000 millimes
Boutiques	NBE 43.500 ; TUN 55.500 TND
Produits	Carnet 22.500 ; Stylo 18.000 ; Classeur 36.000 ; Surligneur 22.500 TND
Clés étrangères	PRAGMA foreign_key_check renvoie []
Second passage du même lot	inserted=0 ; existing=12

Lire l'instruction de transaction et localiser le conflit dans votre code. La somme est utile mais ne remplace pas les clés : un fait pourrait avoir le bon montant et pointer vers la mauvaise boutique. Suivre une vente source permet de vérifier le sens du lien.

PowerShell / Ubuntu · racine du cours

```
docker compose run --rm atelier python chargement.py
docker compose run --rm atelier python verifier.py 05
```

Question après correction

Une nouvelle vente puis un conflit arrivent dans le même lot. Quelle preuve montre que la première écriture a vraiment été annulée ?

CORRIGÉS / UNE IDENTITÉ STABLE

Correction 06 : nouvel arrivage et second passage

Moment	Comptes / état
Lot d'ajout	raw=3, accepted=3, rejected=0, duplicates=0
Premier passage	inserted=2, existing=1
Base après ajout	14 faits, 35 articles, 115 500 millimes
Second passage	inserted=0, existing=3 ; état inchangé

La référence de solutions/atlas/incremental.py appelle catalogues(), lire_csv(), nettoyer() puis charger(). Elle écrit rejets_ajout.csv et doublons_ajout.csv dans le dossier de la base après validation du chargement. Même vides, les journaux portent leur en-tête.

Le premier added lot change la période, qui passe du 5–7 janvier au 5–8 janvier 2026. La fin de période est une valeur calculée ; elle ne doit pas rester bloquée dans le texte du rapport.

PowerShell / Ubuntu · racine du cours

```
docker compose run --rm atelier python incremental.py
docker compose run --rm atelier python verifier.py 06
```

Question après correction

La ligne déjà chargée est-elle une erreur du CSV ? Expliquer pourquoi elle est accepted dans le lot mais existing dans le chargement.

CORRIGÉS / LE CHIFFRE ET SA PREUVE

Correction 07 : un rapport issu de SQLite

La référence de solutions/atlas/rapport.py interroge la base pour le global et les deux regroupements. Elle renvoie facts, quantity, amount_millimes, period_start, period_end, shops, products et écrit le HTML à partir de ces valeurs. Les libellés sont échappés avant leur inclusion dans la page.

Rapport final	Attendu
Ventes élémentaires / articles	14 / 35
Montant	115.500 TND
Période observée	2026-01-05 au 2026-01-08
Regroupements	Somme des boutiques = somme des produits = total global.
Origine des valeurs	Requêtes sur la base actuelle, aucune valeur attendue codée en dur.

PowerShell / Ubuntu · racine du cours

```
docker compose run --rm atelier python rapport.py
docker compose run --rm atelier python verifier.py 07
docker compose run --rm atelier python executer.py
docker compose run --rm atelier python verifier.py tout
```

Le programme executer.py est fourni dans projet/ pour assembler les fonctions. Une relance sur une base déjà chargée peut montrer zéro nouvelle vente initiale : c'est normal si les totaux restent stables. Le programme ne remet pas silencieusement la base à zéro.

Question après correction

Choisir un montant du HTML et retrouver sa requête puis une vente qui y contribue. Quelle conclusion sur la marge serait impossible avec ces seules données ?

REPRISE / UN DOSSIER NOUVEAU À CHAQUE FOIS

Créer une reprise indépendante

Pour reprendre après 03, l'outil copie projet/ dans une nouvelle destination, restaure les fonctions 01 à 03 depuis la référence, et exécute ces seules étapes dans cette copie. Il n'efface pas votre tentative. Les fonctions déjà complétées dans votre tentative sont aussi copiées ; vérifiez ce que vous avez déjà construit avant de poursuivre.

Windows natif · racine du cours

```
py -3 outils/jalon.py 03 --destination reprise-03
```

Sous Ubuntu natif : python3 outils/jalon.py 03 --destination reprise-03. Puis ouvrir reprise-03/ dans l'éditeur, placer le terminal dans ce dossier et lancer py -3 verifier.py 03 ou python3 verifier.py 03.

Avec Docker, depuis la racine, saisir sur une ligne : docker compose run --rm -w /workspace atelier python outils/jalon.py 03 --destination reprises/reprise-03. Ensuite, pour le contrôle de cette copie, saisir sur une ligne : docker compose run --rm -w /workspace/reprises/reprise-03 atelier python verifier.py 03.

- Choisir une destination qui n'existe pas : l'outil refuse l'écrasement.
- Après 03 : fichiers nettoyés et journaux présents, aucune base nécessaire.
- Après 05 : base initiale à 99.000 TND ; après 06 : base à 115.500 TND.
- Conserver ancien dossier, journal, erreur et comparaison du correctif.

Un jalon ne s'exécute pas dans le dossier principal

Pour continuer dans la copie Docker, conserver le même -w /workspace/reprises/reprise-03 dans les commandes des étapes suivantes. Sinon vous reviendriez exécuter projet/ par défaut.

DÉFI / RÉUTILISER LA MÉTHODE

CampusBox : indices gradués

Indice 1 · le périmètre

Un repas non servi ne contribue pas aux ventes servies. Conserver ANNULEE et EN_ATTENTE avec une raison de périmètre, même si leurs autres champs sont valides. Le canal doit survivre au nettoyage et au chargement.

Indice 2 · la petite expérience

Créer trois lignes fictives : une servie au comptoir, une annulée en ligne et la répétition de la première. Prédire 1 fait, 1 exclusion hors périmètre et 1 doublon. Ajouter un canal FAX et vérifier une raison différente.

Indice 3 · le contrat de code

Adapter une fonction à la fois. Faire produire le traitement de status/channel puis le tester avant de changer le SQL. Demander au chargeur de comparer aussi channel lors d'un identifiant répété. Un canal différent change le contenu de la vente.

Le modèle peut garder channel dans fact_sales ou utiliser une dimension. Le rapport doit pouvoir regrouper les montants par point et par canal. Le choix de forme doit préserver un fait par line_id et une somme unique.

Avant la correction chiffrée

Calculer les totaux du lot initial à partir des lignes servies, puis prédire l'effet des quatre lignes d'ajout. Vérifier les comptes de qualité avant la somme d'argent.

DÉFI / À CONSULTER APRÈS VOS PRÉDICTIONS

CampusBox : résultats et limites de correction

Contrôle	Résultat de référence
Lot initial	14 = 9 retenues + 1 doublon + 4 non retenues.
Non retenues initiales	R010 ANNULEE ; R011 quantité zéro ; R012 menu inconnu ; R013 EN_ATTENTE.
Base initiale	9 faits ; 17 portions ; 120.500 TND.
Ajout	4 reçues : 3 retenues + 1 rejet R014 canal FAX.
Chargement ajout	2 nouvelles ; 1 existante (L006).
Base finale	11 faits ; 20 portions ; 140.000 TND.
Rejeu ajout	0 nouvelle ; 3 existantes ; R014 encore non retenue.

Regroupement	Initial TND	Final TND
NBE / TUN	64.500 / 56.000	70.000 / 70.000
COMPTOIR / EN_LIGNE	67.000 / 53.500	81.000 / 59.000

La période initiale va du 2 au 3 février 2026 ; la période finale se termine le 4 février. Deux campus à montant final égal n'impliquent ni volumes, ni coûts, ni rentabilité égaux. Chaque membre doit encore pouvoir expliquer une exclusion et une preuve de rejeu.